

Pairwise Testing

Demonstração acadêmica — explicação completa do projeto, do código e da técnica

Projeto Java + Maven + JUnit 5 | Pacote `com.denariusdata.pairwise`

1. O que é Pairwise Testing (a ideia)

A maioria dos defeitos de software não depende de um único parâmetro, nem de três ao mesmo tempo — eles surgem da **combinação de dois** parâmetros. Estudos clássicos (NIST) mostram que cobrir todos os **pares** de valores detecta a grande maioria das falhas.

O Pairwise (ou *all-pairs*) aproveita isso: em vez de testar **todas** as combinações possíveis, ele seleciona o **menor conjunto de casos que garante que todo par de valores apareça ao menos uma vez**.

O nosso exemplo: 3 parâmetros, cada um com 3 valores possíveis.

Parâmetro	Valores possíveis
pagamento	Cartao, Pix, Boletto
frete	Normal, Expresso, Economico
perfil	Novo, Recorrente, VIP

- **Teste exaustivo:** $3 \times 3 \times 3 = 27$ casos (testa tudo).
- **Teste Pairwise:** apenas **9 casos** cobrem todos os pares.

Redução de **66%** no número de testes, mantendo a cobertura de pares.

2. Estrutura do projeto (o que é cada arquivo)

Pairwise_Testing/	
├ pom.xml	configuração do Maven (Java 17, JUnit 5)
├ pict-model.txt	entrada da ferramenta PICT
├ pict.exe	a ferramenta PICT (Microsoft)
├ mvnw.cmd / .mvn/	Maven embutido (roda sem instalar Maven)
├ COMANDOS.txt	comandos prontos para copiar
├ README.md	documentação
└ src/	
├ main/java/.../CheckoutService.java	o sistema testado (SUT)
└ test/java/.../	
├ CheckoutPairwiseTest.java	os 9 casos (destaque)
└ CheckoutExhaustiveTest.java	os 27 casos (contraste)

A divisão `main` (programa real) e `test` (verificações) é uma convenção obrigatória do Maven. A pasta `target/` é criada automaticamente ao rodar os testes e guarda os relatórios — é descartável.

3. O sistema testado: CheckoutService.java

É uma calculadora de total de compra. Em testes, chamamos o programa avaliado de **SUT — System Under Test** (sistema sob teste). O método principal é:

```
double calcularTotal(double subtotal, String pagamento, String frete, String perfil)
```

Ele recebe 4 informações e devolve o total. As regras de cálculo:

Regra	Efeito
Frete	Normal +20 Expresso +40 Economico +10
Desconto por perfil	VIP ×0,90 Recorrente ×0,95 Novo ×1,00
Desconto por pagamento	Boleto ×0,98 Pix ×0,97 Cartao ×1,00
Valor inválido	lança <code>IllegalArgumentException</code> (erro)

A conta final é: `(subtotal + frete) × fatorPerfil × fatorPagamento`.

O bug proposital (o coração da demonstração)

Existe uma "chave" no código:

```
static final boolean BUG_PROPOSITAL = false;
```

Quando ela vale `true`, o código soma **+999** ao total **somente** quando `pagamento = "Pix"` e `frete = "Expresso"`:

```
if (BUG_PROPOSITAL && pagamento.equals("Pix") && frete.equals("Expresso")) {
    total += 999;
}
```

Por que esse bug é especial? Ele só aparece na combinação de **DOIS** parâmetros específicos. É exatamente o tipo de falha de integração que o Pairwise foi feito para detectar — e que um teste de um parâmetro por vez não pegaria.

4. O teste Pairwise: CheckoutPairwiseTest.java (destaque)

Executa **apenas 9 casos**. Esses 9 foram escolhidos de forma que todos os pares de valores apareçam ao menos uma vez. Em Java, a lista é assim:

```

Arguments.of("Cartao", "Normal", "Novo"), // Caso 1
Arguments.of("Cartao", "Expresso", "Recorrente"), // Caso 2
Arguments.of("Cartao", "Economico", "VIP"), // Caso 3
Arguments.of("Pix", "Normal", "Recorrente"), // Caso 4
Arguments.of("Pix", "Expresso", "VIP"), // Caso 5 <- pega o bug
Arguments.of("Pix", "Economico", "Novo"), // Caso 6
Arguments.of("Boleto", "Normal", "VIP"), // Caso 7
Arguments.of("Boleto", "Expresso", "Novo"), // Caso 8
Arguments.of("Boleto", "Economico", "Recorrente") // Caso 9

```

As anotações do JUnit 5 que fazem a mágica:

- `@ParameterizedTest` — o teste roda **várias vezes**, uma por caso.
- `@MethodSource("casosPairwise")` — pega os casos daquela lista de 9.
- O nome de cada execução fica `"Caso 5: Pix + Expresso + VIP"`.

Em cada caso, o teste verifica dois **invariantes de negócio**:

```

assertTrue(total > 0, "O total deve ser positivo");
assertTrue(total <= 140, "O total não pode passar de 140");

```

`assertTrue` = "afirme que isto é verdade; se não for, o teste falha". O limite de 140 vem de 100 (produto) + 40 (frete máximo); os descontos só reduzem o valor.

A prova: quando o bug soma +999, o total passa de 1100 — estoura o limite de 140 — e o teste falha no **Caso 5 (Pix + Expresso)**. Ou seja, **apenas 9 testes** foram suficientes para detectar o bug de integração.

5. O teste exaustivo: CheckoutExhaustiveTest.java (contraste)

Serve só para comparar tamanho. Usa três laços `for` aninhados que geram **todas as 27 combinações**:

```

for (String pagamento : PAGAMENTOS)
  for (String frete : FRETES)
    for (String perfil : PERFIS)
      // gera 1 teste para cada combinação -> 3 x 3 x 3 = 27

```

Usa `@TestFactory` (forma do JUnit de criar vários testes em tempo de execução). Conclusão visível no console: **27 testes** contra os **9** do Pairwise — mesma cobertura de pares, 3× mais casos.

6. A ferramenta PICT (de onde vêm os 9 casos)

O **PICT** (*Pairwise Independent Combinatorial Testing tool*, da Microsoft) gera automaticamente o conjunto mínimo de casos. A entrada é o arquivo `pict-model.txt` :

```
pagamento: Cartao, Pix, Boletto
frete:      Normal, Expresso, Economico
perfil:     Novo, Recorrente, VIP
```

Rodando `.\pict.exe pict-model.txt /r:5`, ele imprime exatamente os mesmos 9 casos que estão no teste — provando que não foram inventados.

Fluxo no mundo real (duas etapas separadas):

1. **Fora do Java:** descreve-se o modelo → o PICT gera os casos num arquivo.
2. **No Java:** os casos alimentam o teste. Com muitos casos (ex.: 200), não se digita à mão — o teste lê o arquivo do PICT direto (`@CsvFileSource`).

No nosso projeto os 9 casos foram digitados à mão só por serem poucos e ficarem visíveis na tela durante a apresentação.

Sobre o `/r:5`

O PICT usa uma **heurística** e parte de uma **semente** (número inicial). O `/r:5` fixa essa semente em 5, tornando o resultado **repetível**. Dependendo da semente o PICT pode gerar 9 ou 10 casos (ambos válidos; 9 é o mínimo ótimo). A semente 5 produz a solução de 9 casos idêntica ao teste.

7. Como rodar

Pré-requisito único: **Java 17+**. O Maven vem embutido (`mvnw.cmd`).

```
# 1) rodar todos os testes (verde: 9 + 27 = 36)
.\mvnw.cmd test

# 2) gerar os 9 casos com o PICT
.\pict.exe pict-model.txt /r:5

# 3) demonstrar o bug:
#   abrir CheckoutService.java, trocar BUG_PROPOSITAL para true, salvar
.\mvnw.cmd test          # -> falha no "Caso 5: Pix + Expresso"
#   voltar BUG_PROPOSITAL para false e salvar
```

8. Resumo para a defesa

Pergunta	Resposta curta
O que é Pairwise?	Testar todos os pares de valores em vez de todas as combinações.
Por que 9 e não 27?	9 já cobre todos os pares; 27 é testar tudo. -66%.
Como sei que 9 bastam?	Um bug em Pix+Expresso foi detectado pelos 9 testes (Caso 5).
De onde vêm os 9 casos?	Da ferramenta PICT, a partir do pict-model.txt.
Dá para fazer à mão?	Sim em casos pequenos; em sistemas grandes usa-se ferramenta.
O que é JUnit / Maven?	JUnit roda os testes; Maven organiza e executa o projeto.